

### Exercise 1. (0.5 point)

We all have used services like Netflix, Amazon, and YouTube. These services use very sophisticated systems to recommend the best items to their users to make their experiences great. Now, let's us do a small research. How services like Netflix, Amazon, and YouTube recommend items to the users?

To answer such questions, in the first step, we will visit these three websites.

1/ <https://www.netflix.com/pl/>

2/ <https://www.amazon.com/>

3/ <https://www.youtube.com/>

In the second step, try to figure out what data sources are collected and further utilized by these three services. Now, try to fill out the Table below

| On-line Service | Log Files | Content-Based | User's Explicit Preferences | Other? |
|-----------------|-----------|---------------|-----------------------------|--------|
| Netflix         |           |               |                             | ...    |
| Amazon          |           |               |                             | ...    |
| YouTube         |           |               |                             | ...    |

### Exercise 2. (1 point)

Considering the answers given in Exercise 1, now your task is to design on the high level a recommendation system for newly developed web portal. The Architect decided to use log files as the data source to feed up the recommendation system.

In essence, the recommender system essentially addresses the information matching problem to better match user information with item information.

Moreover, please briefly describe each component of your system, and name the system as well each of its component.

### Exercise 3. (1 point)

By definition, collaborative filtering is a type of recommendation engine that uses both user and item data. More specifically, ratings from individual users on individual items. This way, items are recommended based on the ratings from other users, thus, collaborative.

This data can be represented in a utility matrix, with one axis being users and one axis being items. The goal of collaborative filtering recommendation engines is to fill in the gaps in a utility matrix since not every user has rated every item, and then output the top-rated, previously-unrated items as recommendations.

New, let's consider this simple dataset, where U1, U2, U3, and U4 are different users, while 2, 3, 4 and 5 represent four different items.

|   | A  | B  | C  | D  |
|---|----|----|----|----|
| 1 | U1 | U2 | U3 | U4 |
| 2 | 4  | 2  | 3  | 5  |
| 3 | 3  | 2  | 4  | 2  |
| 4 |    | 4  | 5  | 4  |
| 5 | 3  | 2  | 4  | 4  |

Now, you can use any available tool, including Excel, R or Python, and compute the cosine similarity of user U1 with other users.

|   |     |      |      |      |      |
|---|-----|------|------|------|------|
| 8 | cos | U2   | U3   | U4   | Suma |
| 9 | U1  | 0,65 | 0,76 | 0,83 | 2,24 |

I used the following formula:

```
=SUMA.ILOCZYNÓW($A$2:$A$5;B2:B5) / (PIERWIASTEK(SUMA.KWADRATÓW(B2:B5))
*PIERWIASTEK(SUMA.KWADRATÓW($A$2:$A$5)))
```

If you are not interested in using Excel, you can use Python (it is even recommended for you).

```
from scipy import spatial
U1 = [4, 3, 0, 3]
U2 = [2, 2, 4, 2]
U3 = [3, 4, 5, 4]
U4 = [5, 2, 4, 4]
CosU1U2 = 1 - spatial.distance.cosine(U1, U2)
CosU1U3 = 1 - spatial.distance.cosine(U1, U3)
CosU1U4 = 1 - spatial.distance.cosine(U1, U4)
print('Cosine similarity between User1 and User2 is ', CosU1U2)
print('Cosine similarity between User1 and User3 is ', CosU1U3)
print('Cosine similarity between User1 and User4 is ', CosU1U4)
```

[Here](#), you can learn more about the cosine distance.

What is weighted average of ratings for item 3 in case of the User1?

To compute the unknown rating, I used the following formula:

```
= (B9*B4+C9*C4+D9*D4) / E9
```

So our predicted rating for U1 and I3 is 4.34! Also note, different similarity metrics would give slightly different results.

Now, take a look at the given dataset below. As you can see, there is 10 users {u1, ..., u10} and ten items {i1, ..., i10}, and eventually we have also a user, named as Kate.

| Items/ Users | u1 | u2 | u3 | u4 | u5 | u6 | u7 | u8 | u9 | u10 | Kate |
|--------------|----|----|----|----|----|----|----|----|----|-----|------|
| i1           | 99 | 12 | 50 | 95 | 49 | 15 | 11 | 94 | 19 | 66  |      |
| i2           | 10 | 80 | 55 | 12 | 65 | 90 | 89 | 11 | 88 | 43  | 8    |
| i3           | 80 | 13 | 65 | 85 | 54 | 15 | 17 | 89 | 21 | 45  |      |
| i4           | 50 | 87 | 75 | 49 | 49 | 95 | 93 | 48 | 98 | 34  |      |
| i5           | 40 | 10 | 45 | 39 | 34 | 17 | 19 | 42 | 9  | 21  | 90   |
| i6           | 34 | 93 | 48 | 32 | 65 | 94 | 92 | 30 | 90 | 45  |      |
| i7           | 99 | 10 | 56 | 88 | 67 | 12 | 16 | 92 | 11 | 75  |      |
| i8           | 23 | 80 | 52 | 12 | 48 | 85 | 94 | 16 | 97 | 32  |      |
| i9           | 10 | 87 | 59 | 9  | 52 | 90 | 98 | 12 | 89 | 12  | 8    |
| i10          | 17 | 90 | 62 | 21 | 59 | 93 | 92 | 25 | 86 | 55  |      |

Kate has already evaluated only three items. Now, take a closer look at the dataset. What is the main issue with a user Kate?

Now, compute the cosine similarity between Kate and other 10 users, and eventually compute the missing data. Now, in your opinion, in what extent the applied method has produced the reliable results?

#### **Exercise 4. (2 points)**

Now your job is to solve the cold start problem. By definition, the recommender systems face a problem in recommending items to users in case there is very little data available related to the user or item. Therefore, let us consider once again the dataset below.

| Items/ Users | u1 | u2 | u3 | u4 | u5 | u6 | u7 | u8 | u9 | u10 |
|--------------|----|----|----|----|----|----|----|----|----|-----|
| <b>i1</b>    | 99 | 12 | 50 | 95 | 49 | 15 | 11 | 94 | 19 | 66  |
| <b>i2</b>    | 10 | 80 | 55 | 12 | 65 | 90 | 89 | 11 | 88 | 43  |
| <b>i3</b>    | 80 | 13 | 65 | 85 | 54 | 15 | 17 | 89 | 21 | 45  |
| <b>i4</b>    | 50 | 87 | 75 | 49 | 49 | 95 | 93 | 48 | 98 | 34  |
| <b>i5</b>    | 40 | 10 | 45 | 39 | 34 | 17 | 19 | 42 | 9  | 21  |
| <b>i6</b>    | 34 | 93 | 48 | 32 | 65 | 94 | 92 | 30 | 90 | 45  |
| <b>i7</b>    | 99 | 10 | 56 | 88 | 67 | 12 | 16 | 92 | 11 | 75  |
| <b>i8</b>    | 23 | 80 | 52 | 12 | 48 | 85 | 94 | 16 | 97 | 32  |
| <b>i9</b>    | 10 | 87 | 59 | 9  | 52 | 90 | 98 | 12 | 89 | 12  |
| <b>i10</b>   | 17 | 90 | 62 | 21 | 59 | 93 | 92 | 25 | 86 | 55  |

A user, named John, has just created a new account. Your task is to propose a method which is able to recommend the most accurate items. Naturally, it is possible for you to enrich the above dataset by other attributes which enhance your method. A tip: try to think in terms of time, place and environment, therefore a context is something you should definitely consider.

#### **Exercise 5. (0.5 point)**

Consider by yourself the following three users. User A is a fan of the Harry Potter series and watch only such kinds of movies on the Internet. Based on this information, develop a simple data model which should have at least five attributes.

Solution. To solve this problem, one can take advantage of the tabular form. Let's name our table Movies. A single movie can be describe by the following attributes:

| Name | Year of Production | Place of production | Length | Director | Category |
|------|--------------------|---------------------|--------|----------|----------|
|      |                    |                     |        |          |          |

In your opinion, is this piece of information enough for the recommendation system? If not, what other attributes should be included. Try to justify your answer.

Please, provide the [logical](#) and [physical](#) data model. In our opinion, does SQL queries can be used by the recommendation engine in a relatively simple scenario?

**Exercise 6. (5 points, Author: dr inż. Aleksandra Karpus)**

Implement collaborative filtering recommender that predicts user rating for an item. Test different configurations (e.g. different number of nearest neighbors) and evaluate them by usage of (implemented by yourself) MAE and RMSE. You can perform cross or hold-out validation. Implement also naive algorithm that always predict a user rating for an item as an average of this item ratings. Evaluate it in the same way as collaborative filtering algorithm with the same measures. Compare results. What can you say about them?

For testing your algorithms use the MovieLens dataset:

- <https://grouplens.org/datasets/movielens/100k/>

Please, pack your code and report and upload a single zip file.

Your report should contain a table with results obtained with both algorithms as well as conclusions that you have made based on it.